Information Retrieval Assignment 1 - Quad Query

This project was implemented to preprocess the Cranfield collection, build an inverted index, and support simple Boolean retrieval using AND/OR queries. The main implementation is in `QuadQuery_pa1.py`, the stemmer is in `QuadQuery_porter.py`, and the project instructions are in `README.md`.

1. Objective

The main goal of the assignment was to perform the basic preprocessing steps required for information retrieval:

- tokenize the text,
- normalize the tokens,
- remove stopwords,
- stem words,
- create an inverted index,
- and then answer Boolean queries over the indexed data.

The collection used was the Cranfield dataset file, and only the title and abstract sections of each document were considered for indexing.

2. Methodology

The program follows a standard IR pipeline.

First, the document collection is parsed from the Cranfield format. Each document starts with a marker such as `.I`, `.T`, and `.W`. The code reads each document, stores its ID, and keeps only the title and abstract text. The other fields, such as `.A` and `.B`, are ignored intentionally as per the assignment requirements.

After that, the text is tokenized using a regular expression that extracts alphabetic words only. This ensures that punctuation, numbers, and other non-letter symbols are not treated as tokens. The tokens are then converted to lowercase and short tokens of length 1 are removed. This helps reduce noise in the vocabulary.

Then stopwords are removed using the list from `stopwords.txt`. This step is done before stemming because the stopword file is in lowercase and unstemmed form. If stemming were done first, the stopwords would not match correctly.

Finally, each remaining token is passed through a Porter stemmer. The stemmer reduces the word to its root form, so words like “running”, “runner”, and “runs” can be mapped to similar stems for retrieval. This makes matching more effective in a boolean retrieval system.

3. Indexing

Once preprocessing is complete, the code builds an inverted index. In this index, every unique term is mapped to the list of document IDs containing that term. These postings lists are sorted in ascending order, which makes later query processing efficient.

The index file stores:

- the vocabulary size and maximum document ID on the first line,
- and then each term with its list of document IDs.

This allows fast retrieval when a user queries multiple terms.

4. Boolean Search

For query processing, the query is also preprocessed in the same way as the documents. The terms are tokenized, lowercased, stopwords removed, and stemmed before searching.

The code accepts queries in the form:

- `term1 AND term2`
- `term1 OR term2`

For Boolean retrieval, the postings lists are merged using two-pointer logic:

- `AND` uses an intersection of two sorted lists,
- `OR` uses a union of two sorted lists.

This approach is more efficient than using general set operations because the lists are already sorted, and the merge process works in linear time relative to the lengths of the lists.

5. Program Files and Their Roles

The project consists of the following main files:

- `QuadQuery_pa1.py`: main program for preprocessing, indexing, and query search
- `QuadQuery_porter.py`: Porter stemming logic
- `README.md`: project overview, usage instructions, and execution details
- `stopwords.txt`: list of stopwords removed during preprocessing

- `cran.all.1400`: Cranfield dataset used as input

6. Example Commands

The README explains how to run the program. The basic workflow is:

python3 QuadQuery_pa1.py preprocess cran.all.1400 stopwords.txt

python3 QuadQuery_pa1.py index QuadQuery_processed.all

python3 QuadQuery_pa1.py search QuadQuery_cran.index stopwords.txt "aerodynamic AND wing"

python3 QuadQuery_pa1.py search QuadQuery_cran.index stopwords.txt "boundary OR shock"

These commands create the processed file, build the index, and run Boolean queries over the document collection.